



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Sebastian Alexander Villeda Reyna **Carné:** 1032625

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

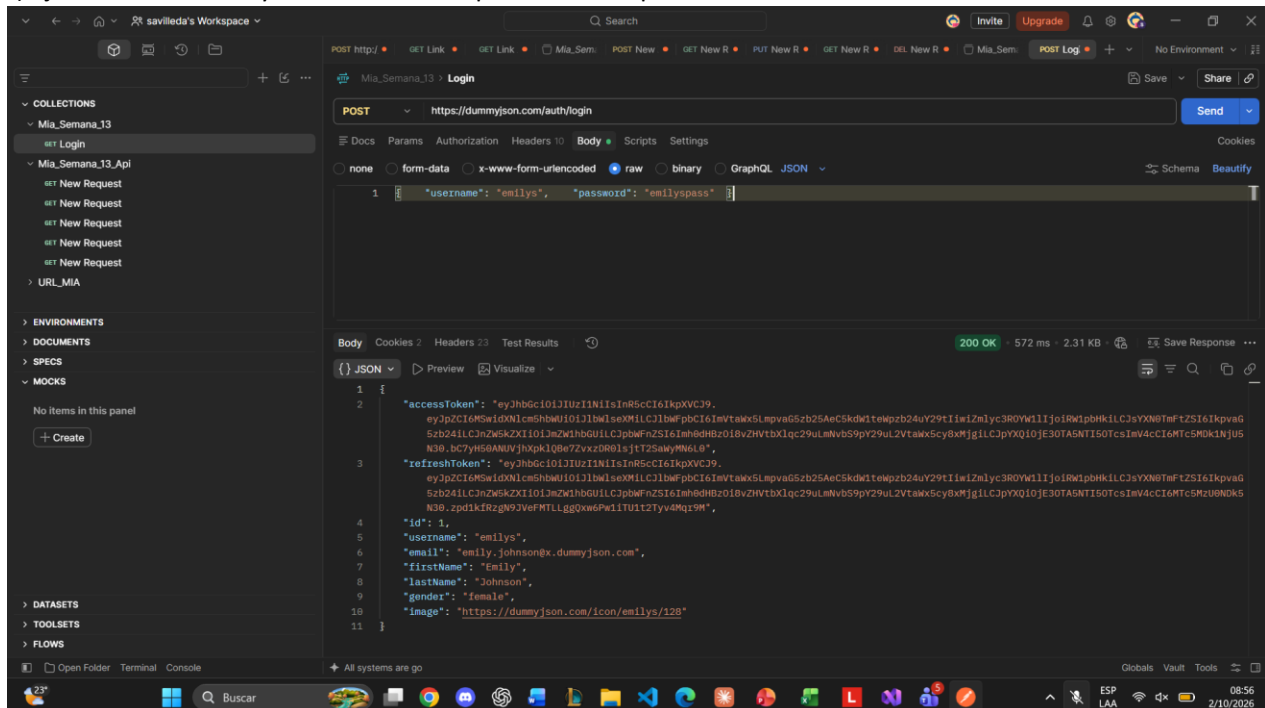
1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: `https://dummyjson.com/auth/login`
- En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

```
{ "username": "emilys", "password": "emilyspass" }
```

- Ejecute la solicitud y localice en la respuesta el campo accessToken.

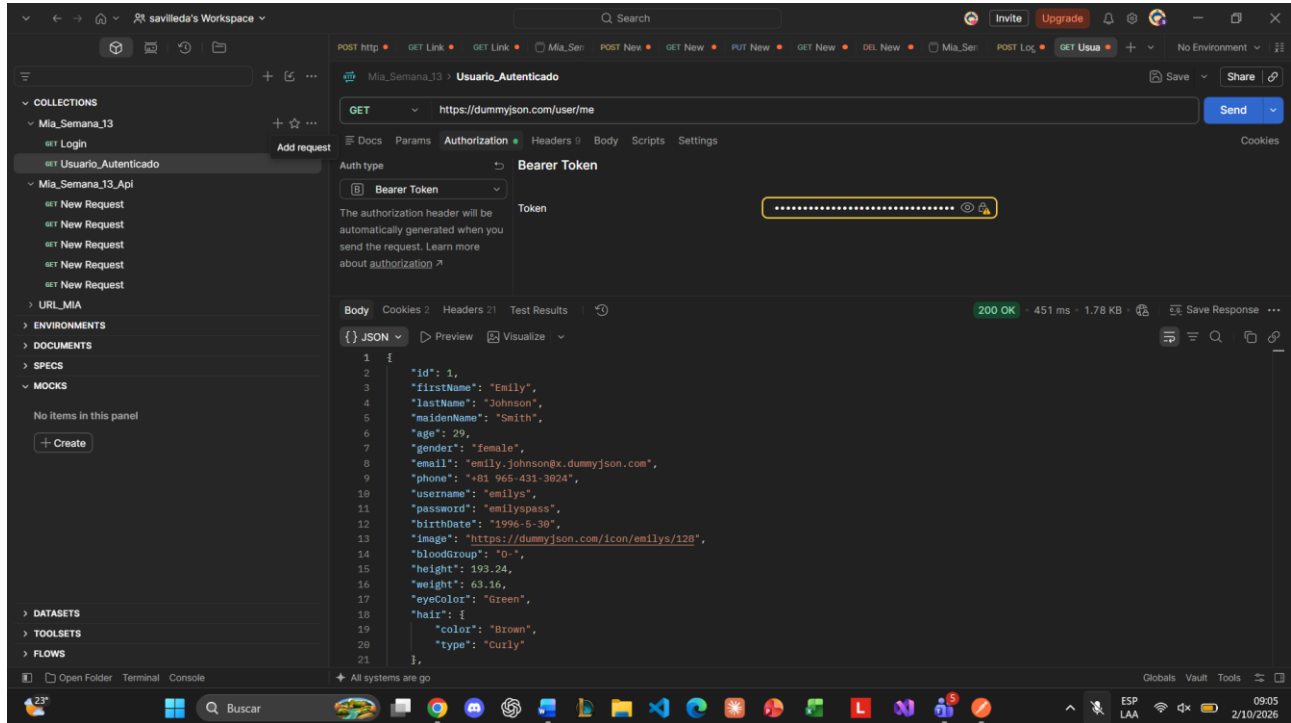


2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`



- d) En Authorization seleccione Bearer Token.
- e) Copie el accessToken obtenido en la solicitud Login.
- f) Ejecute la solicitud.
- g) Verifique que la API devuelve la información del usuario autenticado.

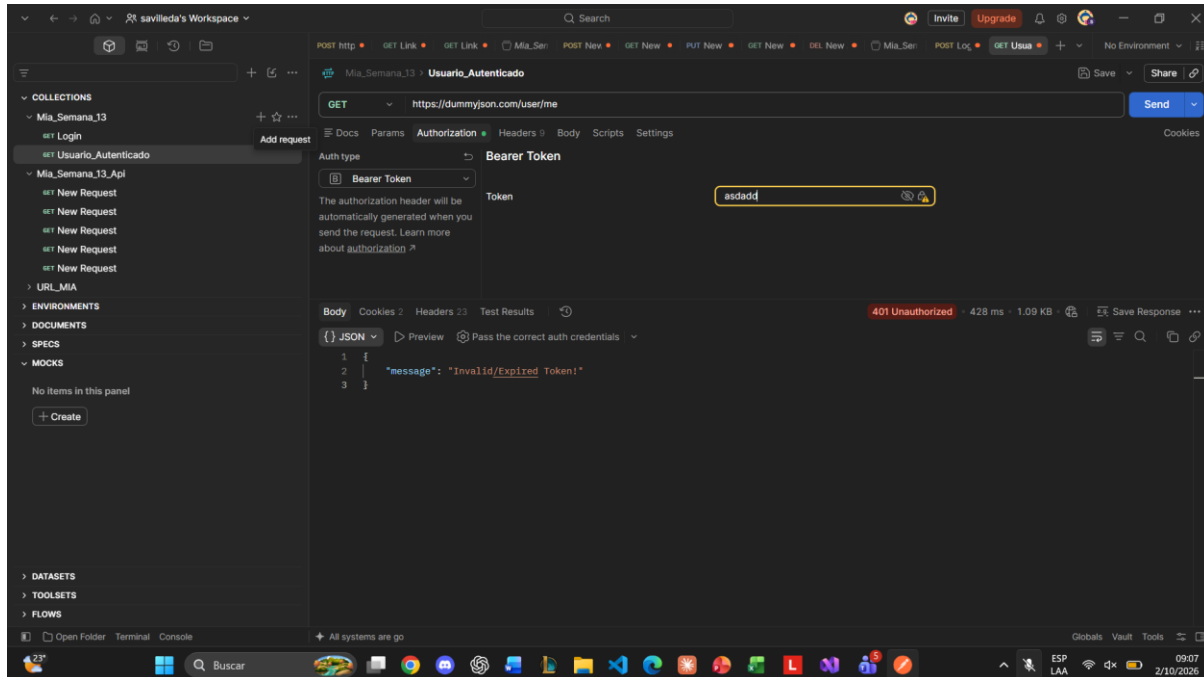


3. Comprobar qué ocurre sin autenticación

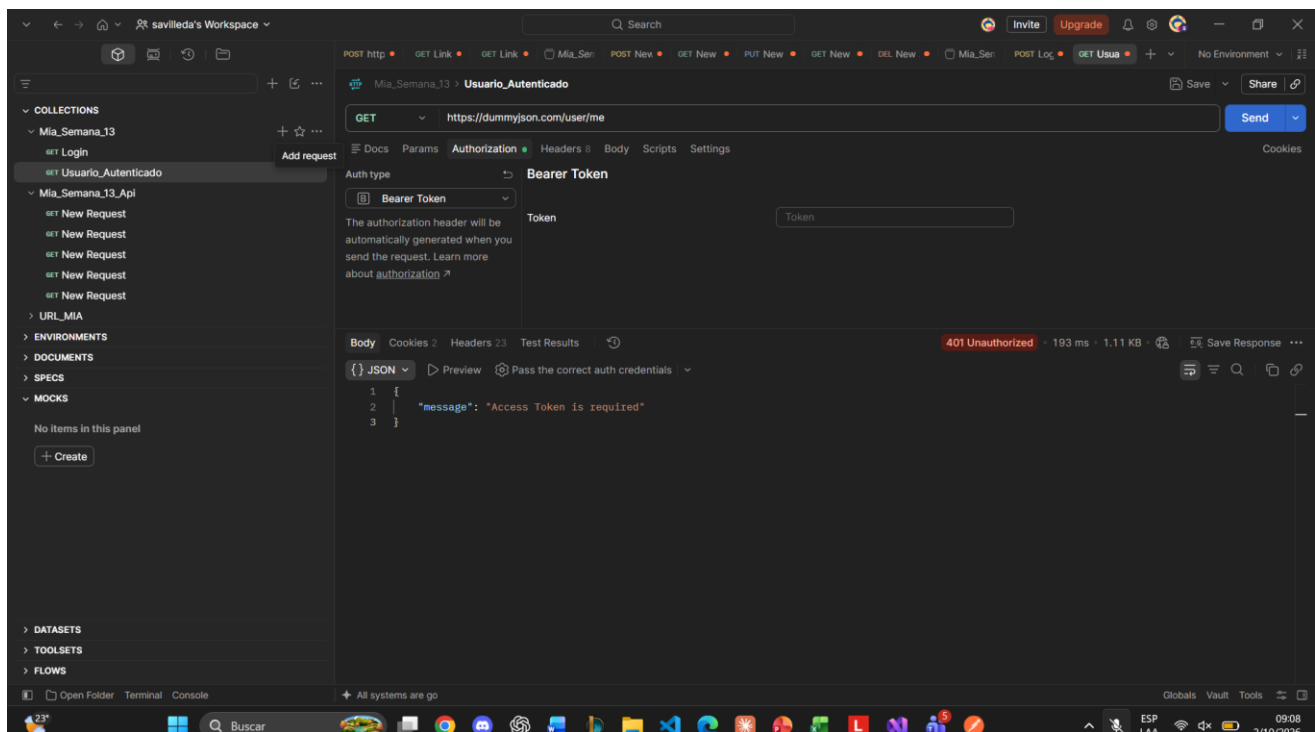
- a) Duplique la solicitud Usuario autenticado.
- b) Quite el token.
- c) Ejecute nuevamente la solicitud.
- d) Compare la respuesta con la solicitud autenticada.
- e) Registre el código HTTP y explique la diferencia.



Si da error 401, significa que el token que se coloco no es el que era, por lo que hay que poner el correcto para utilizarlo.



Si da 404 es porque no hay un token colocado, el 404 es porque no encontró lo que se estaba buscando, esa es la diferencia entre el 404 y 401.



4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.



- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

Que no tenga variable ambiente si no que tenga un archivo .env

la variable que debemos utilizar tiene que ir aquí en el .env

AZURE_STORAGE_CONNECTION_STRING

El .env nunca se sube

**El .env.production reemplaza los valores con el mismo nombre reemplazando la variable y ese si se sube
git ignore al .env**

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- f) No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: **AZURE_STORAGE_CONNECTION_STRING**
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



```
1 using Azure;
2
3
4 const string containerName = "mia-archivos";
5 const string connectionStringVariable = "AZURE_STORAGE_CONNECTION_STRING";
6
7 LeerArchivoDotEnv(connectionStringVariable);
8 string? connectionString = Environment.GetEnvironmentVariable(connectionStringVariable);
9 if (string.IsNullOrEmpty(connectionString))
10 {
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026\Semana_12\lab2> Remove-Item Env:AZURE_STORAGE_CONNECTION_STRING -ErrorAction SilentlyContinue
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026\Semana_12\lab2> Remove-Item Env:AZURE_STORAGE_CONNECTION_STRING -ErrorAction SilentlyContinue
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026\Semana_12\lab2> Remove-Item Env:AZURE_STORAGE_CONNECTION_STRING -ErrorAction SilentlyContinue
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026\Semana_12\lab2> Remove-Item Env:AZURE_STORAGE_CONNECTION_STRING -ErrorAction SilentlyContinue
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026\Semana_12\lab2> dotnet run
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
Selecciona una opción: 2

Nombre                                     Tamaño (bytes)
-----
El contenedor está vacío.

Presiona una tecla para volver al menú...
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
Selecciona una opción: 1
```

Repository secret added.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

[New repository secret](#)

Name	Last updated
AZURE_STORAGE_CONNECTION_STRING	now



The screenshot shows the Visual Studio Code editor with a project named 'url_mia_2026'. The Explorer sidebar on the left shows the file structure, including folders for 'Semana_12' and 'Semana_13', and files like '.gitignore', 'Lab12.cs', 'MIA_AzureBlo...', 'MIA_Laboratori...', 'README.md', and 'README.txt'. The main editor window displays the content of the '.gitignore' file in the 'Semana_13 > lab2' directory. The file contains the following content:

```
Semana_13 > lab2 > .gitignore
1 bin/
2 obj/
3 .vs/
4 *.user
5 *.suo
6 *.useroscach
7 *.sln.docstates
8
9 # Archivos locales de secretos y configuración
10 appsettings.Local.json
11 appsettings.*.Local.json
12 .env
13 .env.*
14 !.env.production
15 *.secret
16 *.secrets
17 secrets/
18
```

The Terminal window at the bottom shows the execution of Git commands:

```
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026> git commit -m "fix readme semana 13"
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 360 bytes | 360.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/savilleda/url_mia_2026
   a079bad..deaca68  main -> main
PS C:\Users\veget\OneDrive\Escritorio\url_mia_2026>
```

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.